

320152841

```
1/1 ▾ + ⓘ 🔍
```

Tilix: Terminal

1: Terminal ▾ AI □

```
newgrp docker
quique@quique:~.->docker compose version
Docker Compose version v5.5.1
quique@quique:~.->sudo snap install postman
postman (v11/stable) 11.71.7 from Postman, Inc. (postman-inc✓) installed
quique@quique:~.->git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco
git
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 2.59 MiB/s, done.
Resolving deltas: 100% (23/23), done.
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

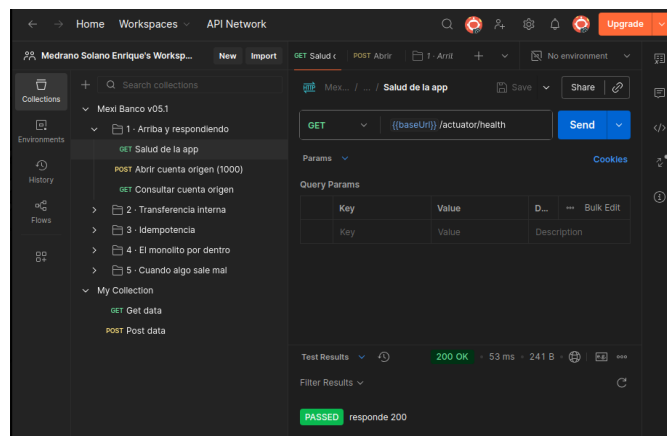
git switch -c <new-branch-name>
```

The screenshot shows a terminal window titled "Tilix Terminal". The top bar contains standard window controls and icons. The main area displays the output of a Docker build command:

```
=> [builder 8/8] RUN --mount=type=cache,target=/var/cache/apt,disable=on mvn clean package -DskipTests
=> [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar
=> exporting to image
=>> exporting layers
=>> exporting manifest sha256:4034a012b372115ee11071a12c11864d290194095998
=>> exporting config sha256:7f227002d21f08aae776638f5c15f49377d69c5af0032e
=>> exporting attestation manifest sha256:32c4c0a02cc2c157d977f12c05516cd41
=>> exporting manifest list sha256:ff5ef84df29be373dd7b2bbc363760dbadd029e58
=>> naming to docker.io/library/mexi-banco-app:latest
[+] up 18/18xking to docker.io/library/mexi-banco-app:latest
✔ Image postgres:17-alpine Pulled 6.1
✔ Image mexi-banco-app Built $9.5
✔ Network mexibanco Created 0.1
✔ Volume mexi-banco_mexibanco_datos Created 0.0
✔ Container mexi-banco-db-1 Healthy 6.0
✔ Container mexi-banco-app-1 Started 6.1
```

Below the build logs, there is a prompt `quique@miquique: ((v05.1))mexi-banco->` followed by the command `docker compose ps`. Below this, a table lists the containers:

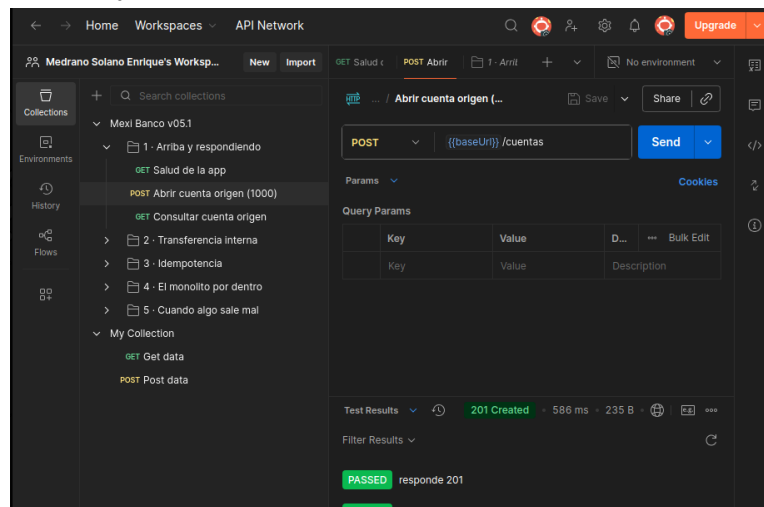
NAME	STATUS	IMAGE	COMMAND PORTS	SERVICE	CREATED
mexi-banco-app-1	Up About a minute (healthy)	mexi-banco-app	"java -jar app.jar"	app	About a m
mexi-banco-db-1	Up About a minute (healthy)	postgres:17-alpine	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp		
nute app	Up About a minute (healthy)	postgres:17-alpine	"docker-entrypoint.s..."	db	About a m
quique@quique: ((v05.1))mexi-banco->			5432/tcp		



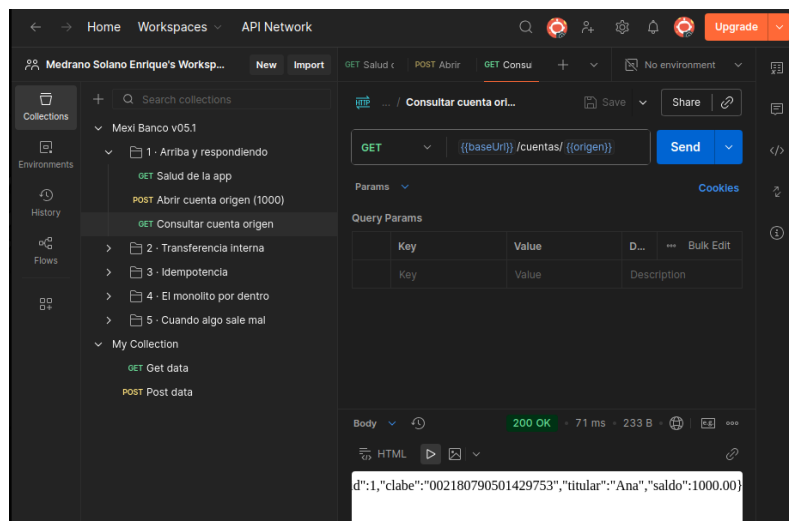
Laboratorio 5 - Sistemas Distribuidos

Medrano Solano Enrique
320152841

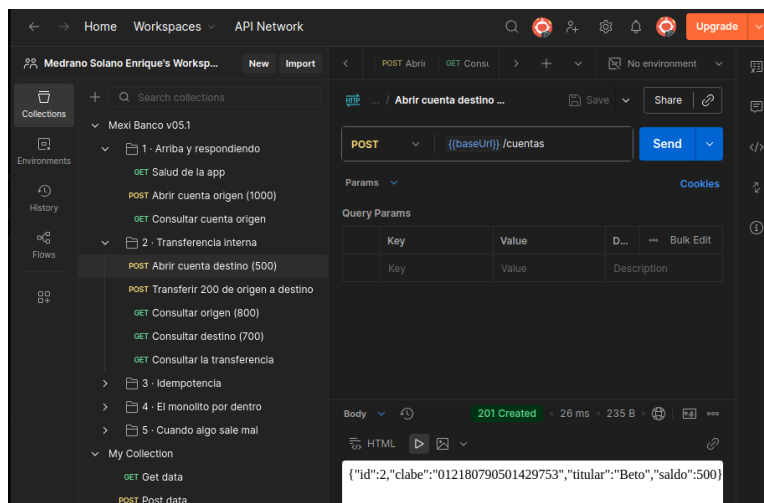
Aquí se comprueba la petición de abrir cuenta origen con 1000 pesos. En donde se hace conexión con el servidor y acepta la petición.



Aquí se hace la consulta a la cuenta origen para saber de quien es cual es su cuenta cable y el saldo disponible.



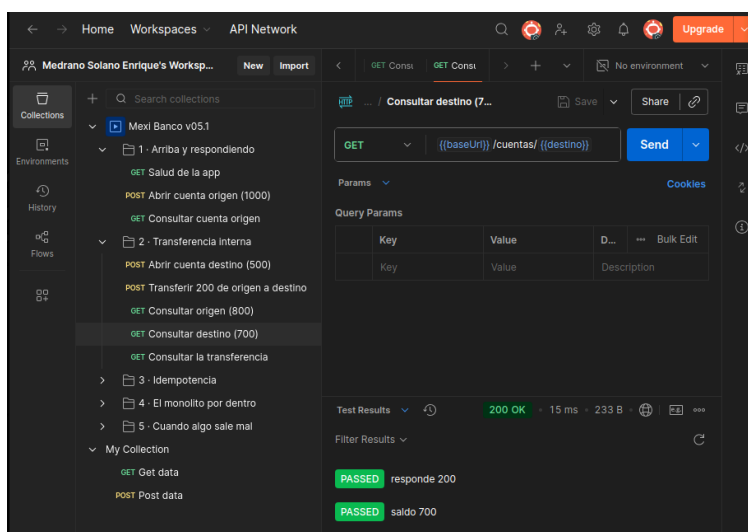
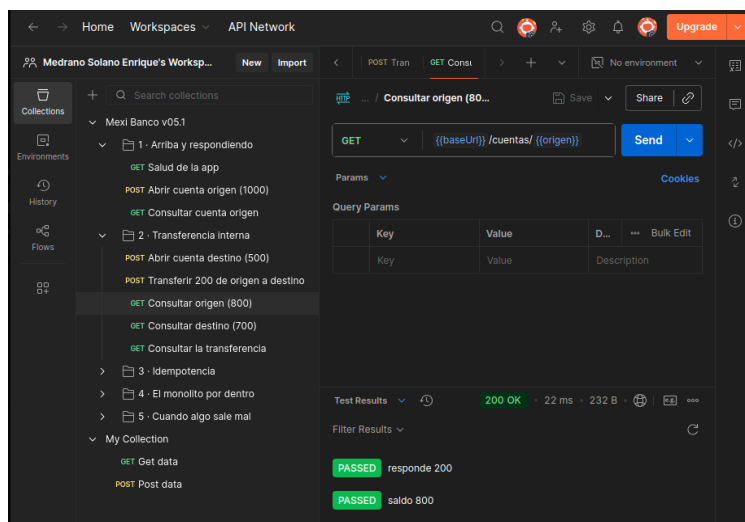
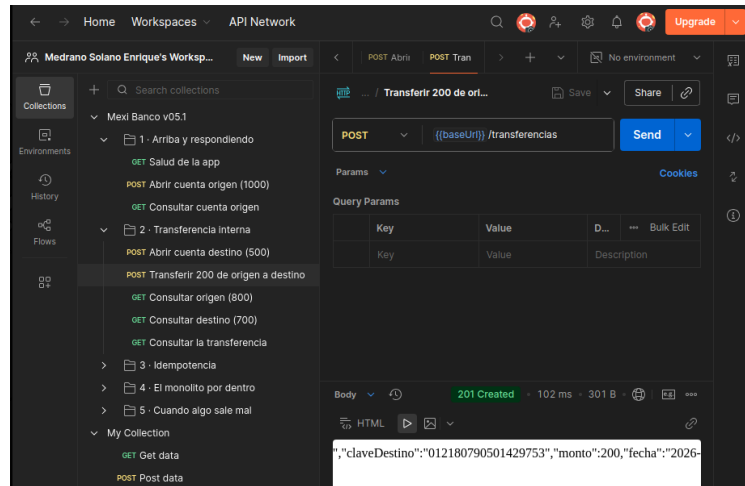
Se hace la petición para la creación de la cuenta destino y poder hacer transacciones más adelante.



Laboratorio 5 - Sistemas Distribuidos

Medrano Solano Enrique
320152841

En las siguientes capturas lo que se le pide al servidor es una petición sobre la transferencia de 200 pesos de la cuenta origen a la cuenta destino, y a su vez se consulta el saldo de la cuenta origen donde ya marcará 800 pesos, y lo mismo con la destino que ahora tendrá 700 pesos. y Finalmente consultar la transferencia con éxito entre las cuentas.

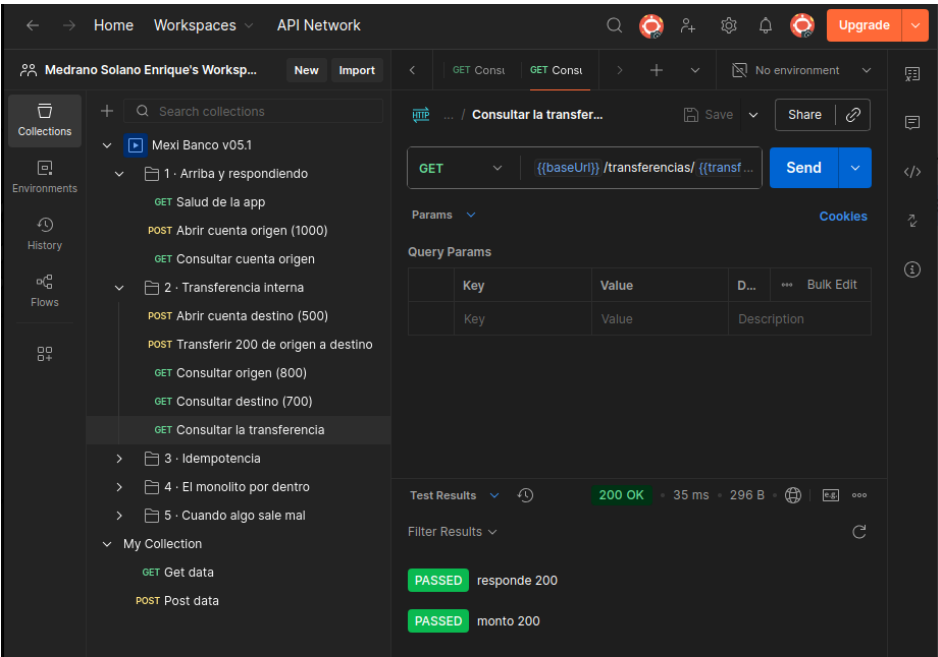
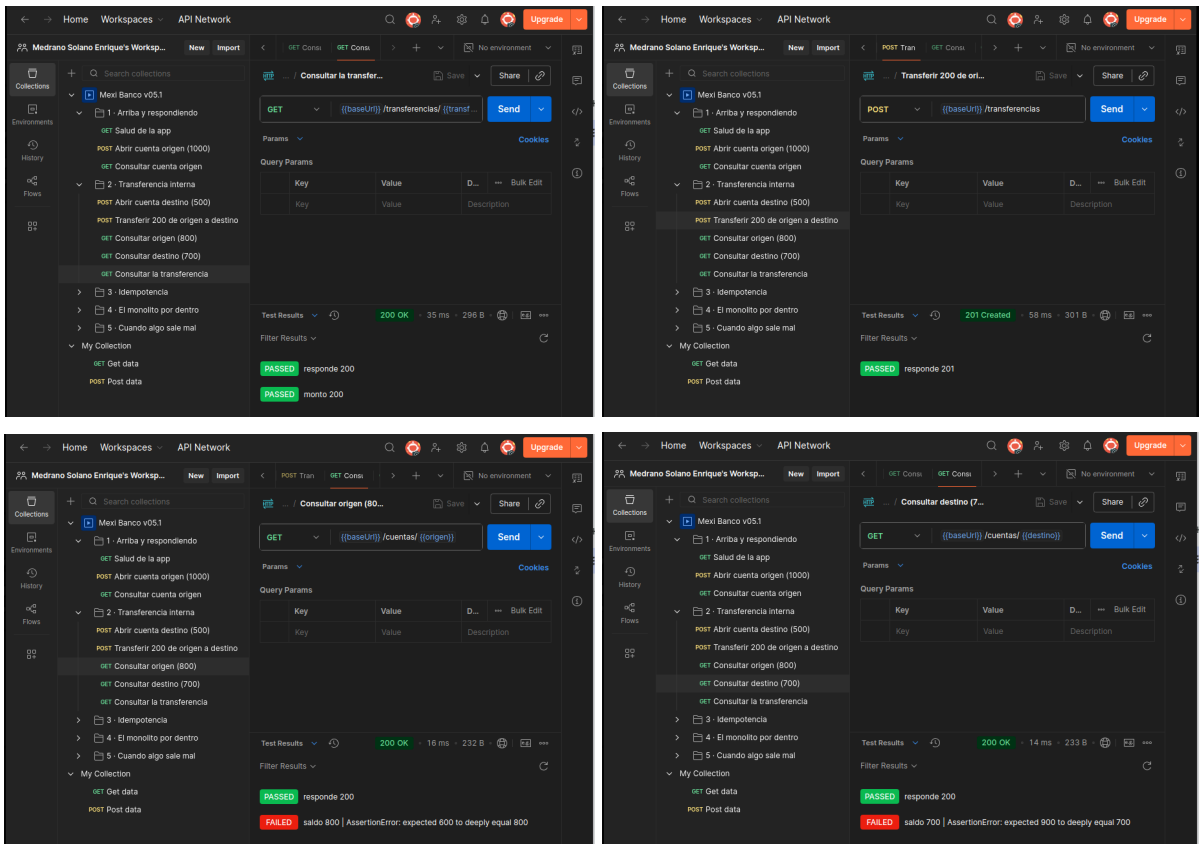


Laboratorio 5 - Sistemas Distribuidos

Medrano Solano Enrique

320152841

Si se vuelve a realizar la transferencia otra vez, no pasará nada, ya que el sistema es robusto para ese tipo de casos y sucede lo que se muestra en las capturas siguientes con los errores pertinentes, por un duplicado de SPEI.



Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

Ahora se procede con una solicitud de SPEI con 50 pesos, y otro intento posterior por la misma cantidad. Pero el sistema también cuenta con la aparte de idempotencia que permite no generar la misma transacción.

The screenshot shows the Postman interface with a workspace named 'Medrano Solano Enrique's Worksp...'. The left sidebar lists collections, including 'Mexi Banco v05.1' which contains several API endpoints. The main panel displays a 'POST' request to 'SPEI de 50 (primer e...' with a body of 'POST' and a URL of '{{baseUri}} /spei'. The 'Test Results' section shows a '201 Created' status with a response of '201'.

Key	Value	D...	Bulk Edit
Key	Value	Description	

Test Results: 201 Created, 484 ms, 339 B. Filter Results: PASSED responde 201.

The screenshot shows the Postman interface with a workspace named 'Medrano Solano Enrique's Worksp...'. The left sidebar lists collections, including 'Mexi Banco v05.1' which contains several API endpoints. The main panel displays a 'POST' request to 'SPEI de 50 (reintent...' with a body of 'POST' and a URL of '{{baseUri}} /spei'. The 'Test Results' section shows a '201 Created' status with a response of '201'.

Key	Value	D...	Bulk Edit
Key	Value	Description	

Test Results: 201 Created, 177 ms, 342 B. Filter Results: PASSED responde 201, PASSED es el MISMO SPEI, no uno nuevo.

Laboratorio 5 - Sistemas Distribuidos

Medrano Solano Enrique

320152841

Aquí se puede observar lo que pasa con el saldo, salta un error sobre el saldo incorrecto por la misma parte de seguridad y de idempotencia.

The screenshot shows the Postman API client interface. The left sidebar displays a collection named 'Mexi Banco v05.1' with several API endpoints. The main panel shows the 'Consultar origen (750, no 700)' endpoint selected. The URL is `GET {{baseUri}}/cuentas/{{origen}}`. The test results section shows a 'PASSED' status for 'responde 200' and a 'FAILED' status for 'saldo 750 | AssertionError: expected 550 to deeply equal 750'.

The screenshot shows the Postman API client interface. The left sidebar displays the same collection. The main panel shows the 'Estado de cuenta del...' endpoint selected. The URL is `GET {{baseUri}}/movimientos?clabe={{origen}}`. The test results section shows a 'PASSED' status for 'responde 200' and a 'PASSED' status for 'un solo SPEI, ENVIADO'. There is also a 'FAILED' status for 'los movimientos suman el saldo (750) | AssertionError: expected 550'.

The screenshot shows the Postman API client interface. The left sidebar displays the same collection. The main panel shows the 'Avisos del origen' endpoint selected. The URL is `GET {{baseUri}}/notificaciones?clabe={{origen}}`. The test results section shows a 'PASSED' status for 'responde 200' and a 'FAILED' status for 'un aviso por la transferencia y uno por el SPEI | AssertionError: expected 550'.

Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

Este marca error 404 porque no ha ocurrido ningún error de CLABE

The screenshot shows the Postman API client interface. The left sidebar displays a collection of API requests under the name 'Medrano Solano Enrique's Worksp...'. The main panel shows a GET request to the endpoint `{{baseUrl}}/cuentas/99999999999...`. The test results at the bottom indicate a '404 Not Found' status with a response time of 13 ms and a body size of 324 B. The test results section shows 'PASSED' and 'responde 404'.

Key	Value	D...	Bulk Edit
Key	Value	Description	

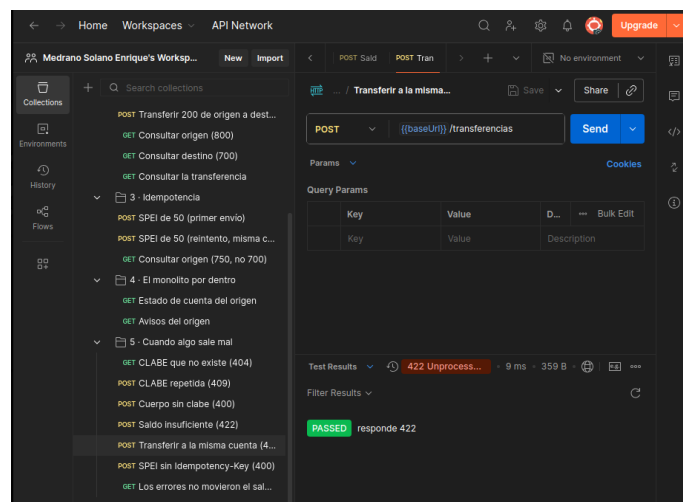
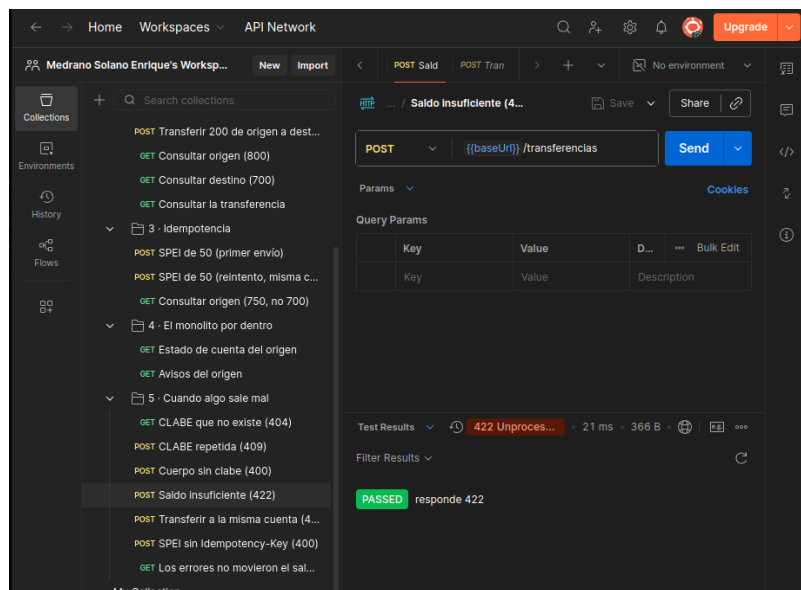
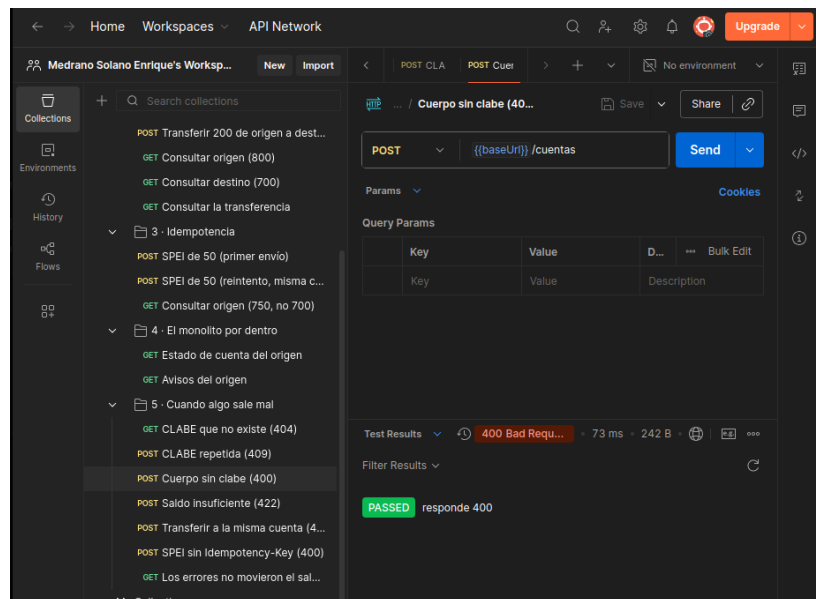
The screenshot shows the Postman API client interface. The left sidebar displays a collection of API requests under the name 'Medrano Solano Enrique's Worksp...'. The main panel shows a POST request to the endpoint `{{baseUrl}}/cuentas`. The test results at the bottom indicate a '409 Conflict' status with a response time of 17 ms and a body size of 318 B. The test results section shows 'PASSED' and 'responde 409'.

Key	Value	D...	Bulk Edit
Key	Value	Description	

Laboratorio 5 - Sistemas Distribuidos

Medrano Solano Enrique
320152841

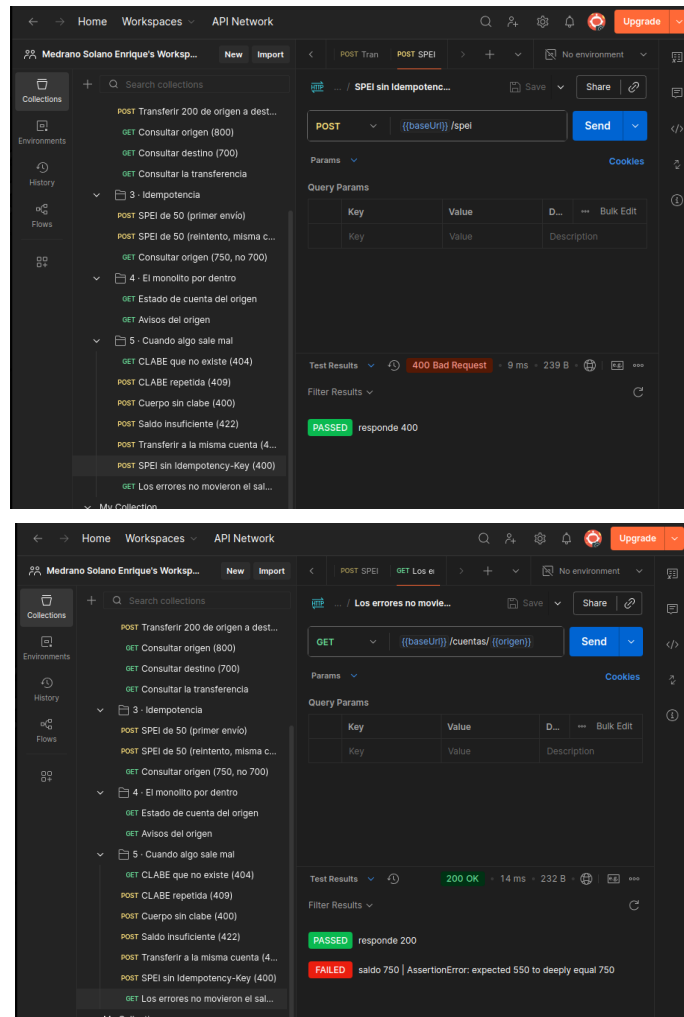
Al igual que este no se ha presentado una CLABE repetida y que no se ha ingresado la CLABE.



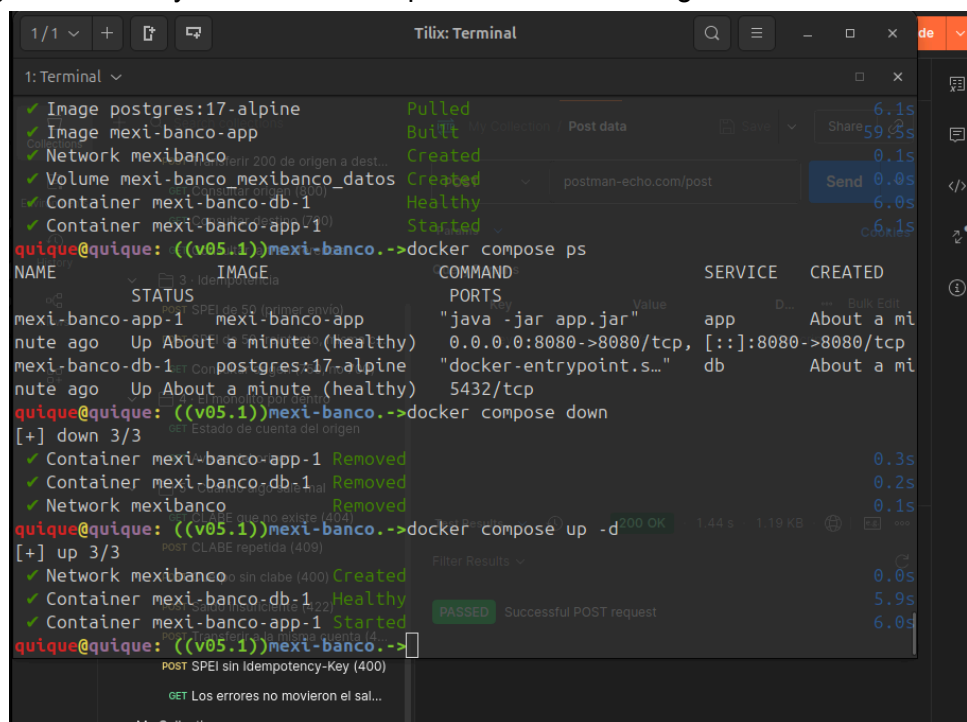
Laboratorio 5 - Sistemas Distribuidos

Medrano Solano Enrique

320152841



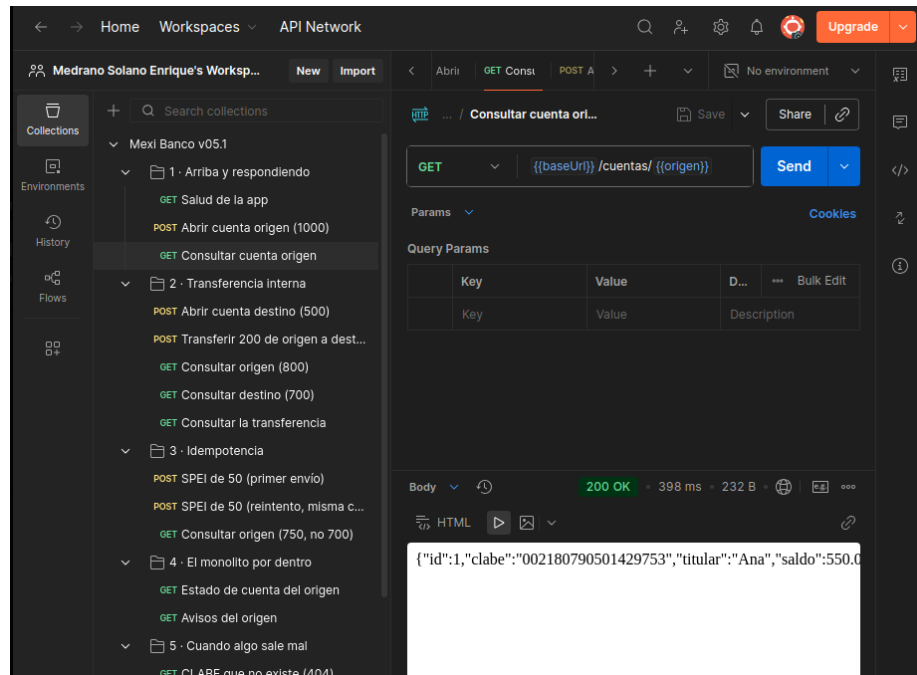
Se apaga el servidor y se vuelve a comprobar si los datos siguen correctos.



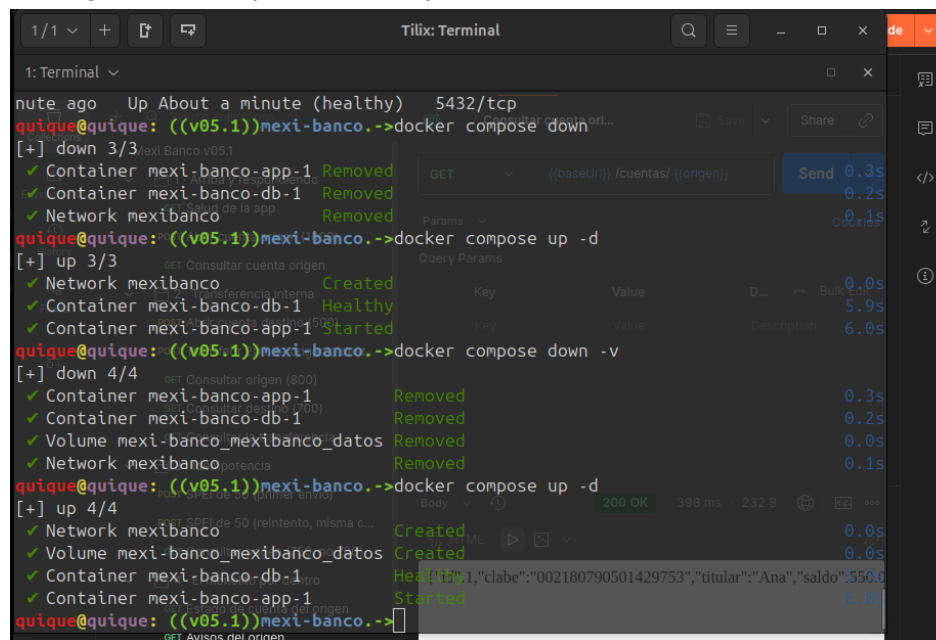
Laboratorio 5 - Sistemas Distribuidos

Medrano Solano Enrique
320152841

Para la prueba se vuelve a verificar un GET en la consulta de la cuenta origen, y se observa que los datos siguen intactos, ya que solo se apago el contenedor, el volumen que es el disco duro sigue sin cambios.

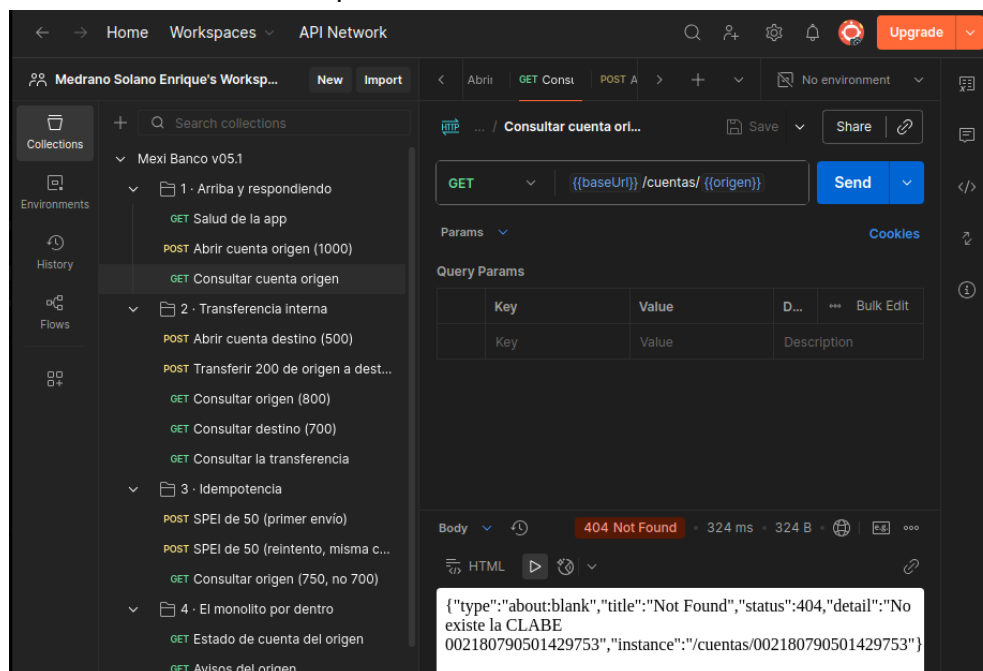


Ahora si se apaga el docker y el volumen y se vuelve a verificar el GET.

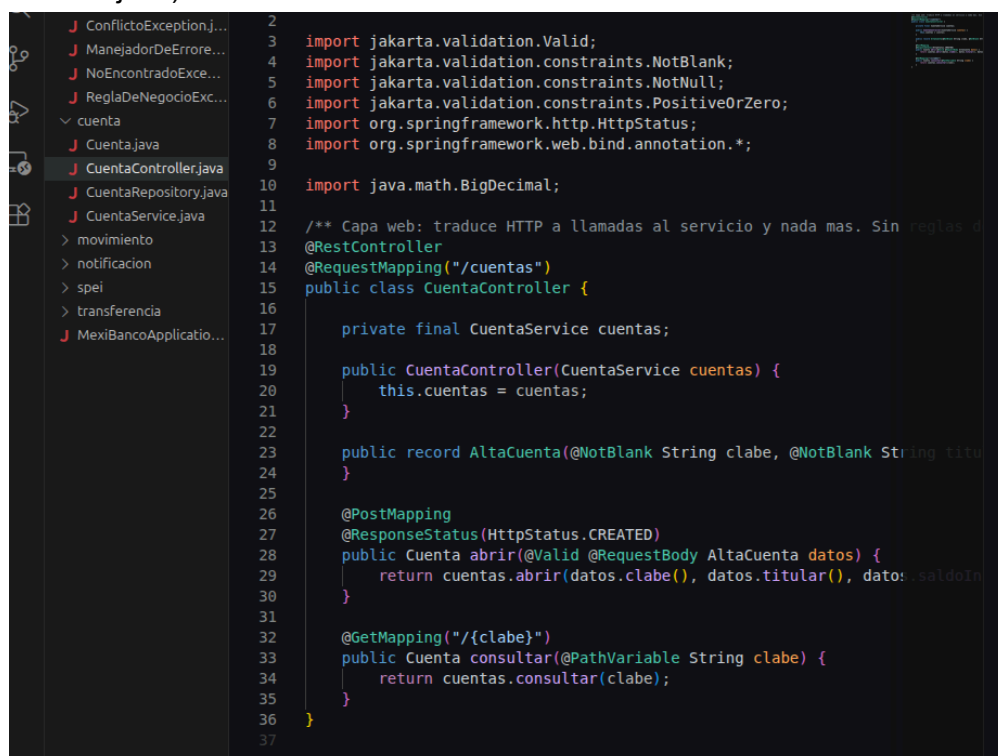


Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

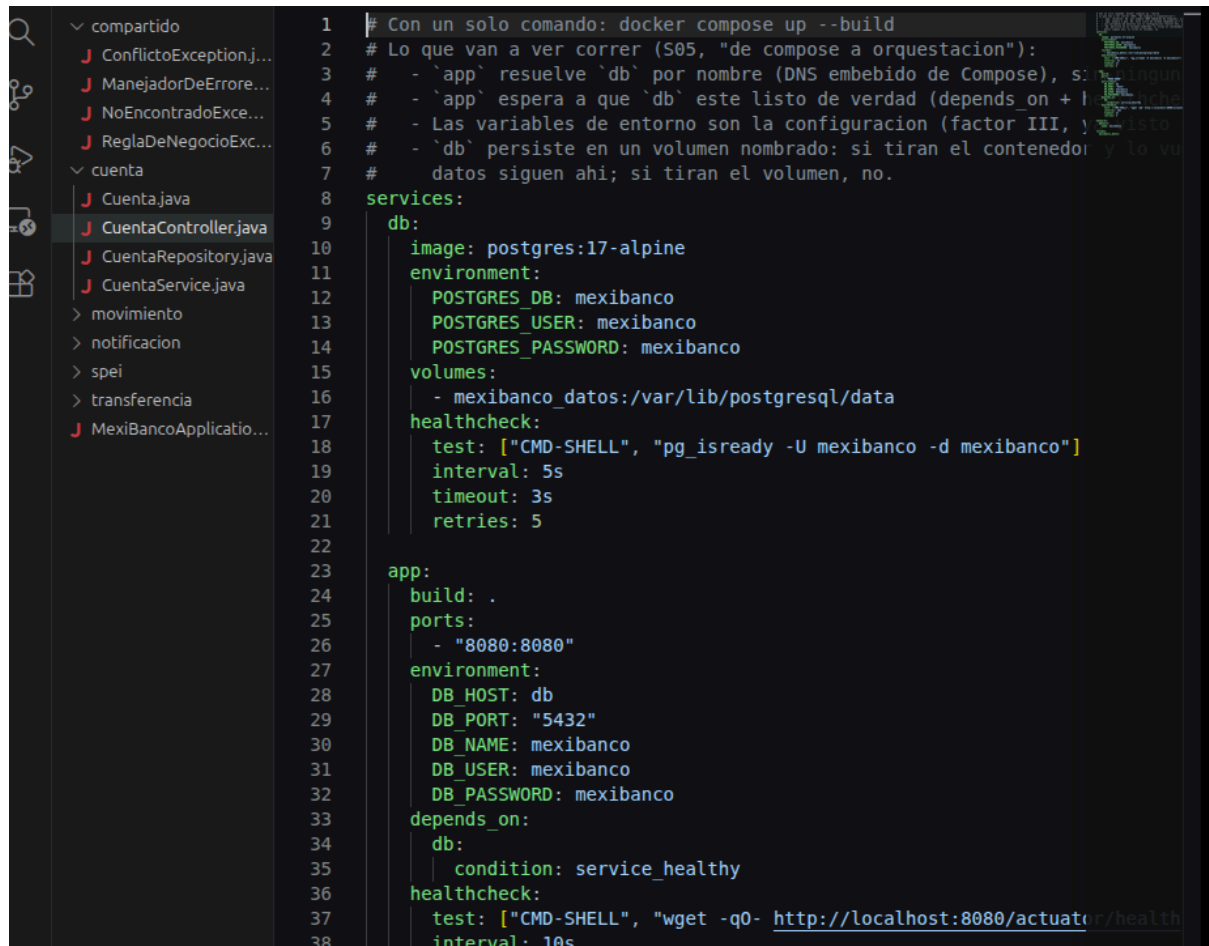
En este caso si manda error, ya que los datos del volumen fueron borrados, por lo que ya no encuentra la cuenta CLABE a la que se intenta acceder.



Se muestra la estructura del proyecto en la captura de CuentaController.java, organizada en paquetes como cuenta, movimiento, notificación, spei y transferencia. En el modulo de cuentas se aprecia la división por capas: Entidad (Cuenta.java), Controlador (CuentaController.java), Repositorio (CuentaRepository.java) y Servicio (CuentaService.java).



El archivo de configuración de Docker Compose define dos servicios: una base de datos (db) usando la imagen de alpine 17 y la aplicación principal (app). Se observa que la aplicación (app) mapea el puerto 8080 y depende directamente del estado de salud de la base de datos mediante la instrucción `depends_on` con la condición `service_healthy`. Además, la base de datos utiliza un volumen nombrado (`mexibanco_datos`) para persistir la información.



```
1  # Con un solo comando: docker compose up --build
2  # Lo que van a ver correr (S05, "de compose a orquestacion"):
3  #   - `app` resuelve `db` por nombre (DNS embebido de Compose), si no hay ningun
4  #   - `app` espera a que `db` este listo de verdad (depends_on + healthcheck)
5  #   - Las variables de entorno son la configuracion (factor III, y esto
6  #   - `db` persiste en un volumen nombrado: si tiran el contenedor y lo vu
7  #     datos siguen ahi; si tiran el volumen, no.
8  services:
9    db:
10     image: postgres:17-alpine
11     environment:
12       POSTGRES_DB: mexibanco
13       POSTGRES_USER: mexibanco
14       POSTGRES_PASSWORD: mexibanco
15     volumes:
16       - mexibanco_datos:/var/lib/postgresql/data
17     healthcheck:
18       test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
19       interval: 5s
20       timeout: 3s
21       retries: 5
22
23    app:
24     build: .
25     ports:
26       - "8080:8080"
27     environment:
28       DB_HOST: db
29       DB_PORT: "5432"
30       DB_NAME: mexibanco
31       DB_USER: mexibanco
32       DB_PASSWORD: mexibanco
33     depends_on:
34       db:
35         condition: service_healthy
36     healthcheck:
37       test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health"]
38       interval: 10s
```